



# TM4 Review Guide for Test 4 on 04.02

Completion requirements

## Decision Trees

- Terms to know:
  - **decision tree**
  - **split** (in learning or using decision trees)
  - **Gini impurity** vs **entropy** (for learning decision trees)
  - **regression trees**
  - **model trees** (don't confuse with **regression trees**! they are different!)
  - **CART (Classification and Regression Tree)**
- You should have a basic understanding of how decision trees are learned (greedy algorithm, recursively selecting most informative node and value to split on).
- You may be given a decision tree and asked to classify an unlabeled point given its feature vector.

## Random Forests

- Terms to know:
  - **random forest** (also known as random decision forest)
  - majority vs. **plurality vote**
  - mean vs. **mode**
  - **black-box model**
  - **interpretable machine learning**
  - **Born-Again Tree Ensembles**
  - **Sampling with (or without) replacement**
  - axis-parallel vs. **oblique linear split**
  - **ensemble learning method**
  - **bagging (Bootstrap AGGregatING)**
  - **AdaBoost (ADaptive BOOSTing)**
  - **weak learner**
  - **weighted majority vote**
- Make sure you are familiar with random forests and how they are created (learned).
- Given a simple example with 3-5 trees, you should be able to predict the class label predicted by a random forest.
- You should be able to provide at least one alternative to axis-parallel splits in decision trees and random forests and to explain the difference.

- You should know what ensemble learning is and be prepared to compare and contrast bagging and boosting. You should also know that random forests are an example of bagging.

### Linear models revisited

- You should know what a **linear model** is (we covered this previously, but it's an important concept in machine learning!).
- You should be prepared to plot a simple linear **decision boundary** for data with just 2 features--or read such a plot to obtain the weights and bias.
- You should know that the decision boundary (50/50 probability boundary) is **orthogonal** (perpendicular) to the weight vector at a point determined by the bias.
- You should understand that in logistic regression:
  - As we travel through feature space **in the direction of the weight vector**, the **probability p of class 1 increases** until it approaches 1. Conversely, the probability  $1-p$  of class 0 (the other class) decreases until it approaches 0.
  - As we travel in the direction **opposite the weight vector**, it is just the opposite: the **probability of class 1 decreases**, and the probability of class 0 increases.
  - If you travel in a direction **parallel to the decision boundary** (i.e., orthogonal to the weight vector), the **probabilities stay the same**.
- You should be able to use the **weights, bias, feature values, and logistic regression formula to estimate probabilities of class membership**, just as we did in class. As a reminder, here are the steps:
  - Plug in values  $x$  to compute  $z = \mathbf{w} * \mathbf{x} + w_0$ : that is, multiply each feature  $x_i$  by its weight  $w_i$ , add these together, and then add the bias  $w_0$ . Note that  $z$  here is just a scalar (real) number.
  - Note that in **linear regression**, the resulting value from the previous step would be our prediction for the target value. But since this is **classification**, we will instead use this value of  $z$  to determine the **decision boundary**.
  - In **logistic regression** we use  $z$  to estimate the probability that the unclassified point  $\mathbf{x}$  belongs to class 1: The formula is:  $\Pr(1|\mathbf{x}) = 1 / (1 + \exp(-z))$ .
  - Note that **exp** here is the inverse of the natural log: Your calculator probably has a button for this (marked  $e^x$  or exp). It just raises 2.71828... to the power of  $z$ .
  - **Summary:**
  - Multiply weights by features and add bias to compute  $z$ .
  - Plug  $z$  into the logistic formula to estimate the probability  $p$  that the feature belongs to class 1.
  - In binary classification, the probability that  $x$  belongs to the other class is  $1-p$ .

- Of course, the computer will usually do all the work for us, but you should know how the process works, and be able to trace it by hand for problems with just 2 or 3 features. Keep in mind, it's not magic, just arithmetic!
- **Discriminate function:** A discriminate function is also a linear model, but it doesn't depend on probabilities. Instead, it just looks at the value of  $z$  that we computed above, but it doesn't plug it into the logistic regression formula. Instead, if  $z \geq 0$ , it classifies the point  $\mathbf{x}$  as class 1. If  $z < 0$ , it classifies the point as class 0.

## Perceptron

- You should know what a **perceptron** is:
  - The model is similar to logistic regression except that it has a **hard decision boundary** rather than a **soft** boundary.
  - The parameters (weights) are learned via the **perceptron learning algorithm**.
  - The algorithm is guaranteed to converge for **linearly separable** datasets.
  - If data are not linearly separable (e.g., **exclusive or [XOR]**), the program will not terminate (or will time out after some preset number of iterations without converging).
  - While one of the simplest models, is the foundation for artificial neural nets and deep learning—topics we will discuss later in the course.
  - Note that simple perceptrons, like logistic regression, employ a **generalized linear model**. In contrast, **multilayer perceptrons (MLP, a.k.a. neural nets)** are most definitely **NOT** linear models!
- A **generalized linear model** is characterized by the presence of an **activation function**.
  - In general, this activation function is **non-linear**.
  - Examples are sigmoid, Heaviside step function, and identity.
  - These are used in logistic regression, perceptron, and linear regression, respectively.
  - We will see more later in the course when we get to neural nets. There are many.

**For all the methods we have covered up to this point in the course (including those covered on previous tests):**

- Be prepared to give a brief explanation of the method.
- Be prepared to discuss advantages and disadvantages.
- Be prepared to compare and contrast the method with any other method we have studied throughout the course.
- Describe the parameters that are learned by the ML algorithm.
- Describe several hyperparameters.

Last modified: Friday, April 11, 2025, 4:54 PM

